

Architecture du module Nutrition

Force Tracker — 11/09/2026. Verification LARGE demandee par Michel **avant** toute reecriture : « *identifier ce qui peut etre simplifie, centralise, supprime, fusionne ou rendu propriétaire unique, **sans casser les fonctionnalites existantes*** ». Suite de DOUANE-NUTRITION.pdf, qui ne regardait que les ecritures. **Tout est trace dans le depot** ; les comptes sont recalculés a chaque generation.

LE DIAGNOSTIC EN UNE PHRASE

Le module n'a pas plusieurs problemes d'architecture : il en a **un seul, repete partout** — **il n'existe aucun objet « aliment » declare**. Chaque porte fabrique sa propre forme, a la main, et la traduit vers la suivante.

Or la forme canonique **existe deja** : {name, kcal, prot, carbs, fat, per100, q, u, portionLabel, portionWeightG}. Elle est simplement **construite a la main a 5 endroits**, sans nom, sans constructeur, sans validateur. *Ce n'est pas une architecture manquante, c'est une architecture non ecrite.*

Conséquence pratique : le gros du gain est mecanique, pas conceptuel — environ cinq fonctions a extraire, et presque aucun comportement a changer.

1. Les portes d'entree : 13, pas 8

Mesure : **11** appels a `_afSetSrc({...})` (une provenance posee) et **8** constructeurs de `_bcNutr`. Le hub `_offRemplirFormulaire` est appele **5** fois.

porte	fonction	ce qu'elle produit	hub ?
scan code-barres	<code>_lookupBarcode</code>	<code>_bcNutr + per100</code>	oui
code-barres tape	<code>_bcManuel -> idem</code>	+ <code>codeDouteux</code>	oui
photo du code-barres	idem (2 chemins)	idem	oui
fiche trouvee SANS valeurs	<code>_bcSansValeurs</code>	provenance sans per100	non
recherche Open Food Facts	<code>_afSuggPrendreOff</code>	per100 normalise	oui
recherche CIQUAL	<code>_afSuggPrendreCiqual</code>	idem	oui, puis ecrase
marques	<code>_afSuggPrendreMarque</code>	+ <code>doute, kcalDerivee</code>	oui, puis ecrase
photo d'etiquette	<code>onFoodLabelFile</code>	per100 normalise	non (copie)
etiquette recopiee a la main	<code>_calAppliquer</code>	<code>_bcNutr</code>	oui
estimation IA (phrase)	<code>estimateFoodAI</code>	totaux seuls, aucun per100	non
Mes aliments -> formulaire	<code>quickFillFood</code>	non normalise	non (copie)
Mes aliments -> ajout direct	<code>quickAddFood</code>	ecrit sans ecran	—
reprise d'un repas	<code>rejouerRepas</code>	ecrit sans ecran	—
reprise depuis le journal	<code>_afSuggPrendreLocale</code>	non normalise	non (copie)
saisie manuelle	les 4 champs	<code>_afSrc = null</code>	—

DECOUVERTE : LE HUB TRAVAILLE POUR RIEN SUR 2 DE SES 5 CLIENTS

`_afSuggPrendreMarque` et `_afSuggPrendreCiqual` appellent le hub — qui pose `_afSetSrc({... etat: 'tel-que-vendu' ...})` — puis **reecrivent immédiatement** `_afSetSrc({... etat: null ...})` juste apres.

Le bloc provenance du hub s'execute **5 fois et se fait ecraser 2 fois**. Ce n'est pas un bug (le resultat est celui qu'on veut, CIQUAL dit son etat dans le nom) : c'est **du code qui ne peut pas etre faux parce qu'il ne sert a rien** — donc un piege pour le suivant qui y ajoutera un champ en croyant servir cinq portes.

2. Les sorties

structure	qui écrit	valide ?
S.foodLog	addFoodEntry · quickAddFood · rejouerRepas	nom + au moins une valeur + geste quantité · rien · rien
S.foodLog (edition)	l'ecran modifier	derive le per100, ne valide pas
S.foodLog (remplace en bloc)	restauration cloud · chargement local · fusion multi-onglets	rien
S.savedFoods	toggleFavFood · _majDefFavori · quickFillFood	rien
S.savedFoods (en bloc)	restauration cloud	rien

Mesure au passage : _fusionListe (state.js) protege 5 listes contre l'ecrasement entre deux onglets ouverts — foodLog en fait partie, savedFoods **non**. Deux onglets, et le dernier persist () efface les etoiles de l'autre. *Petit, mais c'est exactement la famille de pertes que cette fonction existe pour empecher.*

3. Proprietaires reels des regles

regle	proprietaire	etat
redimensionnement de la quantite	_qtyRescale	SAIN — 4 routes, 1 formule. Le modele a suivre
oubli de l'aliment (R15)	_afOublierAliment	SAIN , 13 portes — sauf 1 variable (ci-dessous)
arrondi du pour-100 g	_per100d1	SAIN , 8 portes
masse impossible	_masseImpossibleVals	pure, mais 1 appelant bloquant / 1 en avis
plafond physique des kcal	_kcalImpossibleVals	idem
coherence kcal / macros	_coherenceKcal	affichage seul, jamais bloquant
sec / cuit / pret	_afNoteEtat	SAIN depuis ft-v1191 (3 regex, 1 endroit)
poids du paquet	_bcPaquetG	remis a zero dans openAddFood seulement
derniere quantite	_bcProposerDerniere	SAIN
provenance	_provFood	liste blanche opt-in — 4 oublis recenses
« cette quantite est-elle utilisable ? »	AUCUN	ecrite 6 fois, avec 2 regles differentes
derivation du per100 depuis les totaux	AUCUN	3 blocs identiques
traduction kcal100 vers per100.kcal	AUCUN	4 lignes identiques au caractere pres
la forme d'un aliment	AUCUN	5 constructions a la main

4. Les duplications, chiffrées

duplication	combien	ce que ça coûte
le même per100 traduit depuis _bcNutr	4 lignes identiques	un champ ajoute = 4 endroits à éditer
la dérivation per100 depuis les totaux	3 blocs	ft-v1188 en a corrigé 2 sur 3 (BUGS.md §59)
constructeurs de _bcNutr	8, dont 2 sans _per100d1	le pour-100 g d'une reprise n'est pas normalisé
le hub recopie en variantes locales	3	chacune oublie une liste différente de champs
la forme {name, kcal, ..., portionWeightG}	5 constructions	ajouter portionLabel a demandé 5 éditions (ft-v1186)
le filtre « quantité utilisable »	6 écritures, 2 règles	2 acceptent 'portion', 4 non
états parallèles du même aliment	7 variables	7 cycles de vie à tenir alignés à la main

Les 4 traductions, telles qu'elles dans le dépôt (lignes 1731, 2933, 3483, 3507) — identiques au caractère près :

```
per100:{kcal:_bcNutr.kcal100,prot:_bcNutr.prot100,carbs:_bcNutr.carbs100,fat:_bcNutr.fat100},
per100:{kcal:_bcNutr.kcal100,prot:_bcNutr.prot100,carbs:_bcNutr.carbs100,fat:_bcNutr.fat100},
per100:{kcal:_bcNutr.kcal100,prot:_bcNutr.prot100,carbs:_bcNutr.carbs100,fat:_bcNutr.fat100},
per100:{kcal:_bcNutr.kcal100,prot:_bcNutr.prot100,carbs:_bcNutr.carbs100,fat:_bcNutr.fat100},
```

UNE FUITE LOCALISÉE, PAS ENCORE MESURÉE — DITE PLUTÔT QUE TUE

_afOublierAliment remet à zéro _bcNutr, _bcQtyPose, _bcCategories et les grammes de l'IA — **mais pas** _bcPaquetG, qui n'est nettoyé qu'à openAddFood. Et la pastille du paquet n'est peinte que par _bcProposerPaquet, appelée **uniquement par le hub**.

Donc, en théorie : scanner un produit de 410 g, puis reprendre un aliment par « Mes aliments » **sans fermer l'écran** — la pastille « paquet entier · 410 g » pourrait survivre sur le second produit. **C'est le bug _bcCategories de ft-v1191, sur une autre variable.**

Ca se mesure en un témoin. **Je ne l'ai pas fait** : la consigne était de ne rien corriger. *Je le signale localisé et non vérifié, plutôt que de l'affirmer.*

5. Format interne : il existe, il n'est pas declare

Sept formes differentes aujourd'hui, pour le meme aliment :

forme	ou elle vit	champs
fiche source	reponse Open Food Facts	energy-kcal_100g, proteins_100g...
pour-100 g d'ecran	_bcNutr	kcal100 prot100 carbs100 fat100
provenance	_afSrc	saisie origine sourceId per100 etat q u ...
reference de calcul	_afRef / _efRef	{base:{...}, q, u, src}
ligne enregistree	S.foodLog[]	kcal prot carbs fat + per100{kcal...}
favori	S.savedFoods[]	idem, sans origine/etat
reprise	_afQuickItems[]	idem + fav, origine, sourceId

Le meme nombre s'appelle kcal100 ici et per100.kcal la. Et il existe un FOOD_LOG_V = 1 — un numero de version pour un schema qui n'est ecrit nulle part.

6. Le flux cible minimal

SOURCE (13 portes) -> NORMALISER
1 fonction -> VALIDER
1 fonction -> ALIMENT
1 forme -> DOUANE
1 point -> S.foodLog

Cinq pieces, toutes extraites de code existant — rien de neuf sauf la derniere :

#	piece	remplace	risque
1	_alimentDepuisSource()	les 8 _bcNutr={...} + les 4 traductions	quasi nul — pure extraction
2	_per100Derive(totaux, masse)	les 3 blocs identiques	nul — meme formule
3	_quantiteUtilisable(q, u, {portions})	les 6 ecritures	il faut NOMMER les 2 regles, pas les fusionner
4	le hub rendu obligatoire	les 3 copies locales	moyen — chaque copie a ses omissions
5	la douane avant les 3 push	rien (piece neuve)	le seul vrai chantier

L'ORDRE EST DICTE PAR LE RISQUE, PAS PAR L'ELEGANCE

1, 2 et 3 sont des extractions sans changement de comportement : le critere de reussite est que **chaque temoin actuel reste vert**, sans qu'aucune valeur attendue ne bouge. 4 change un comportement (les copies gagnent ce qu'elles omettaient). 5 est la seule vraie decision produit.

Ce que je ne propose PAS : fusionner _afRef/_efRef/_bcNutr en un etat global, ni reecrire les 13 portes. *Le benefice serait esthetique, le risque est reel.* Les portes peuvent rester 13 — c'est ce qu'elles **fabriquent** qui doit etre unique.

CE QUE CE DOCUMENT N'EST PAS

Aucun correctif n'a ete ecrit, aucune ligne de production n'a change. Michel a demande une cartographie et des pistes avant de decider.

Et une honnetete sur la methode : les comptes ci-dessus sont recalculés a chaque generation et le generateur **refuse de produire** si la forme du depot ne correspond plus (verifie : les gardes sortent en erreur). Mais **la lecture de ce que ces comptes signifient reste la mienne** — le generateur peut prouver qu'il y a 4 traductions identiques ; il ne peut pas prouver qu'elles *devraient* etre une seule.